

Méthode modale

Régime stationnaire 2D et temporel 1D

SAAB Alyaa

Sommaire

1	Principe de la méthode	2
2	Régime stationnaire 2D	2
	2.1 Grille de calcul	2
	2.2 Discrétisation par différences finies	2
	2.3 Hamiltonien 2D par produit de Kronecker	2
	2.4 Diagonalisation creuse avec ARPACK	3
	2.5 Potentiel utilisé	3
	2.6 Erreurs de troncature	3
3	Régime dépendant du temps 1D	4
	3.1 Grille et état initial	4
	3.2 Potentiel utilisé	4
	3.3 Hamiltonien tridiagonal et décomposition modale	4
	3.4 Évolution temporelle	5
4	Conservation de la norme	5

1 Principe de la méthode

L'idée de la méthode modale est de transformer l'équation de Schrödinger en un problème matriciel. On discrétise l'espace, on construit le hamiltonien sous forme de matrice creuse, et on le diagonalise pour obtenir les états propres ψ_j et les énergies propres E_j .

Ces états propres vérifient l'équation aux valeurs propres :

$$H\psi_j = E_j \psi_j \quad (1)$$

Toute solution peut ensuite s'écrire comme une superposition de ces états :

$$\psi(\mathbf{r}, t) = \sum_j c_j \psi_j(\mathbf{r}) e^{-iE_j t}, \quad c_j = \langle \psi_j | \psi(\mathbf{r}, 0) \rangle \quad (2)$$

La conservation de la norme est exacte puisque $|e^{-iE_j t}| = 1$ pour tout t .

2 Régime stationnaire 2D

2.1 Grille de calcul

On discrétise le domaine $[0, 1]^2$ sur une grille $N \times N$ de points équirépartis. En Python :

```
def build_grid_2d(N):  
    X, Y = np.mgrid[0:1:N*1j, 0:1:N*1j]  
    return X, Y
```

2.2 Discrétisation par différences finies

La dérivée seconde est approchée par :

$$\frac{d^2\psi}{dx^2}(x_i) \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2} \quad (3)$$

Ce qui donne la matrice tridiagonale des différences finies :

$$D = \begin{pmatrix} -2 & 1 & 0 & \dots \\ 1 & -2 & 1 & \dots \\ 0 & 1 & -2 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

2.3 Hamiltonien 2D par produit de Kronecker

En 2D, l'opérateur cinétique est :

$$T = -\frac{1}{2}(D_x \otimes I + I \otimes D_y) \quad (4)$$

Le hamiltonien complet est $H = T + V$, avec V une matrice diagonale contenant les valeurs du potentiel. La matrice obtenue est de taille $N^2 \times N^2$.

La fonction `solve_stationary_2d` construit et diagonalise ce hamiltonien :

```

def solve_stationary_2d(N=200, k=10, x0=0.3, y0=0.3,
                        sigma=0.1, amplitude=1.0):
    X, Y = build_grid_2d(N)
    V = gaussian_potential_2d(X, Y, x0=x0, y0=y0,
                              sigma=sigma, amplitude=amplitude)

    diag = np.ones(N)
    diags = np.array([diag, -2 * diag, diag])
    D = sparse.spdiags(diags, np.array([-1, 0, 1]), N, N)
    T = -0.5 * sparse.kronsum(D, D)
    U = sparse.diags(V.reshape(N**2), 0)
    H = T + U

    eigenvalues, eigenvectors = eigsh(H, k=k, which='SM')
    return X, Y, V, eigenvalues, eigenvectors

```

2.4 Diagonalisation creuse avec ARPACK

La ligne `eigsh(H, k=k, which='SM')` cherche les k paires (E_j, ψ_j) vérifiant $H\psi_j = E_j\psi_j$ en utilisant la méthode de Lanczos (ARPACK). Comme H peut atteindre $22\,500 \times 22\,500$ pour $N = 150$, on ne calcule pas tout — seulement les k niveaux les plus bas.

Pour extraire le mode n sous forme de grille 2D :

```

def get_mode_2d(eigenvectors, N, mode):
    psi = eigenvectors[:, mode].reshape((N, N))
    return psi

```

2.5 Potentiel utilisé

Le potentiel est une gaussienne 2D centrée en (x_0, y_0) :

$$V(x, y) = A \exp\left(-\frac{(x - x_0)^2}{2\sigma^2}\right) \exp\left(-\frac{(y - y_0)^2}{2\sigma^2}\right) \quad (5)$$

```

def gaussian_potential_2d(X, Y, x0=0.3, y0=0.3,
                          sigma=0.1, amplitude=1.0):
    V = amplitude * np.exp(-(X - x0)**2 / (2 * sigma**2)) \
        * np.exp(-(Y - y0)**2 / (2 * sigma**2))
    return V

```

2.6 Erreurs de troncature

Si l'état initial ψ_0 a des composantes sur des modes d'énergie $E_j > E_k$ non calculés, celles-ci sont perdues. L'erreur de troncature s'écrit :

$$\varepsilon_{\text{trunc}} = 1 - \sum_{j=0}^{k-1} |c_j|^2 \quad (6)$$

En pratique, pour $\psi_0 = \sqrt{2} \sin(\pi x)$, les coefficients c_j décroissent rapidement et $k = 70$ modes suffisent pour $\varepsilon < 10^{-6}$.

3 Régime dépendant du temps 1D

3.1 Grille et état initial

On construit la grille 1D et l'état initial :

```
def build_grid_1d(Nx=301):  
    x = np.linspace(0, 1, Nx)  
    dx = 1.0 / (Nx - 1)  
    return x, dx  
  
def initial_state_1d(x):  
    return np.sqrt(2) * np.sin(np.pi * x)
```

L'état initial $\psi_0(x) = \sqrt{2} \sin(\pi x)$ est le premier mode de la boîte sans potentiel, vérifiant les conditions de Dirichlet et normalisé sur $[0, 1]$.

3.2 Potentiel utilisé

Un puits gaussien attractif centré en μ :

$$V(x) = A \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad A < 0 \quad (7)$$

```
def gaussian_potential_1d(x, mu=0.5, sigma=0.05,  
                           amplitude=-1e4):  
    return amplitude * np.exp(-(x - mu)**2 / (2 * sigma**2))
```

3.3 Hamiltonien tridiagonal et décomposition modale

En 1D avec conditions de Dirichlet, le hamiltonien est directement tridiagonal. Les éléments sont :

$$d_i = \frac{1}{\Delta x^2} + V(x_i), \quad e_i = -\frac{1}{2\Delta x^2} \quad (8)$$

La fonction `solve_time_basis` construit la base modale complète :

```
def solve_time_basis(Nx=301, mu=0.5, sigma=0.05,  
                     amplitude=-1e4, n_modes=70):  
    x, dx = build_grid_1d(Nx)  
    psi0 = initial_state_1d(x)  
    Vx = gaussian_potential_1d(x, mu=mu, sigma=sigma,  
                               amplitude=amplitude)  
  
    # Matrice tridiagonale sur les points interieurs  
    d = 1 / dx**2 + Vx[1:-1]  
    e = -1 / (2 * dx**2) * np.ones(len(d) - 1)  
  
    w, v = eigh_tridiagonal(d, e)  
  
    E_js = w[:n_modes]  
    psi_js = np.pad(v.T[:n_modes], [(0, 0), (1, 1)],  
                     mode='constant')
```

```
cs = np.dot(psi_js, psi0)
```

```
return x, psi0, Vx, E_js, psi_js, cs
```

`eigh_tridiagonal` est un solveur optimisé de SciPy pour les matrices tridiagonales symétriques, de complexité $\mathcal{O}(N)$. Les coefficients c_j sont calculés par projection scalaire :

$$c_j = \int_0^1 \psi_j(x) \psi_0(x) dx \approx \sum_i \psi_j(x_i) \psi_0(x_i) \quad (9)$$

3.4 Évolution temporelle

À chaque instant t , la fonction d'onde est recalculée par superposition :

$$\psi(x, t) = \sum_{j=0}^{k-1} c_j \psi_j(x) e^{-iE_j t} \quad (10)$$

```
def psi_t(x, E_js, psi_js, cs, t):
    psi = np.zeros_like(x, dtype=complex)
    for j in range(len(E_js)):
        psi += cs[j] * psi_js[j] * np.exp(-1j * E_js[j] * t)
    return psi
```

La densité de probabilité et la surface 3D sont ensuite calculées :

```
def density_t(x, E_js, psi_js, cs, t):
    psi = psi_t(x, E_js, psi_js, cs, t)
    return np.abs(psi)**2

def density_surface(x, E_js, psi_js, cs, t_vals):
    rho = np.zeros((len(t_vals), len(x)))
    for i, t in enumerate(t_vals):
        rho[i] = density_t(x, E_js, psi_js, cs, t)
    return rho
```

4 Conservation de la norme

La norme est conservée exactement par construction :

$$\|\psi(\cdot, t)\|^2 = \sum_j |c_j|^2 |e^{-iE_j t}|^2 = \sum_j |c_j|^2 = 1 \quad (11)$$

puisque ψ_0 est normé et les ψ_j forment une base orthonormée. En pratique, les erreurs numériques restent inférieures à 10^{-10} .